

Unit-6: Game Playing

Table of Topics and Subtopics

Main Topic	Subtopics	Key Concepts to Understand
6.1 Introduction to Game Playing	Concept of AI in games, types of games, characteristics of game environments	How AI agents play strategic games
6.2 Game Tree Representation	Game tree structure, nodes, branches, terminal states, utility values	Representing game states using trees
6.3 Minimax Search Procedure	Minimax concept, maximizing player, minimizing player, algorithm steps	Optimal decision-making in adversarial games
6.4 Alpha-Beta Pruning	Concept of pruning, alpha value, beta value, pruning conditions	Optimizing minimax search by reducing nodes
6.5 Refinements of Game Tree Search	Move ordering, heuristic evaluation functions	Improving efficiency of search
6.6 Iterative Deepening	Concept of iterative deepening search, depth limits	Combining depth-first search with breadth exploration

6.1 Introduction to Game Playing

Game Playing is an important application of Artificial Intelligence where an AI system learns to **make optimal decisions in competitive environments**. In these environments, two or more players compete, and each player tries to **maximize their chances of winning while minimizing the opponent's chances**.

AI game-playing programs analyze possible moves and predict future outcomes using **search algorithms and evaluation strategies**. According to the syllabus section for Unit-6, game playing focuses on **Minimax search, Alpha-Beta pruning, refinements, and iterative deepening** used in strategic games.

1. Concept of AI in Games

Meaning : In AI, a game is considered a **search problem where the system must explore different possible moves to find the best strategy**.

An AI game-playing system:

- analyzes possible moves
- predicts opponent responses
- chooses the optimal action

The AI agent uses **game trees and search algorithms** to determine the best move.

Example : Consider **Tic-Tac-Toe**.

Possible moves:

Current Board

X | O | X

O | X |

| | O

The AI system evaluates possible next moves and chooses the move that **leads to the highest chance of winning**.

Goal of AI in Games

Objective	Explanation
Win the game	choose best strategy
Predict opponent moves	anticipate opponent actions
Optimize decisions	select move with highest benefit

2. Types of Games in AI

Games in Artificial Intelligence are classified based on **environment characteristics and number of players**.

1. Deterministic vs Stochastic Games

Type	Description	Example
Deterministic	outcome determined by current state	chess
Stochastic	randomness involved	backgammon

2. Perfect Information vs Imperfect Information

Type	Description	Example
Perfect Information	all players know entire game state	chess
Imperfect Information	players have incomplete information	poker

3. Two-Player vs Multi-Player Games

Type	Description	Example
Two-player	two competing agents	chess
Multi-player	more than two players	multiplayer board games

4. Zero-Sum vs Non-Zero-Sum Games

Type	Description	Example
Zero-sum	one player's gain equals another's loss	chess
Non-zero-sum	multiple players may benefit	cooperative games

Examples of AI Game Applications

Game	AI Application
Chess	strategic decision making
Go	deep search algorithms
Tic-Tac-Toe	basic game tree search
Checkers	heuristic evaluation

3. Characteristics of Game Environments

Game environments in AI have specific characteristics that affect how AI agents make decisions.

1. Adversarial Environment

In games, players **compete against each other**.

Example:

- player tries to maximize score
- opponent tries to minimize it

2. Sequential Decisions

Game decisions are **sequential**, meaning each move affects future states.

Example: Move 1 → Move 2 → Move 3

Each step changes the game situation.

3. Strategic Planning

Players must plan **several moves ahead** to achieve victory.
AI uses search algorithms to evaluate **future possibilities**.

4. Finite State Space

Games have a **limited number of possible states**, although this number can be very large.
Example:

Game	Possible States
Tic-Tac-Toe	~255,000
Chess	~10^120 possible positions

4. How AI Agents Play Strategic Games

AI agents play strategic games using **search and evaluation techniques**.

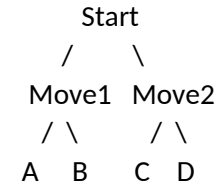
Step-by-Step Process

Step	Description
1	represent game states
2	generate possible moves
3	create game tree
4	evaluate possible outcomes
5	select best move

Game Tree Concept

A **game tree** represents possible moves in a game.

Example:



Each branch represents a possible move.

Strategy Used by AI

AI uses algorithms such as:

Algorithm	Purpose
Minimax	optimal decision making
Alpha-Beta pruning	reduce search complexity
Iterative deepening	improve search efficiency

These methods help AI evaluate **future game states efficiently**.

Example: Chess AI

Chess AI evaluates:

- piece positions
- possible moves
- opponent strategies

Then selects the move with the **highest winning probability**.

Real-World AI Game Systems

AI System	Game
Deep Blue	chess
AlphaGo	Go
OpenAI Five	Dota 2

These systems demonstrate how AI can perform **advanced strategic reasoning**.

Importance of Game Playing in AI

Game playing is important because it helps researchers study:

AI Concept	Application
Search algorithms	game trees
Decision making	optimal moves
Strategy planning	long-term planning

Game environments are useful for testing **intelligent decision-making algorithms**.

Summary

Concept	Meaning
Game Playing	AI decision making in competitive environments
Types of Games	deterministic, stochastic, zero-sum, imperfect information
Game Environment	adversarial, sequential, strategic
AI Strategy	search algorithms and evaluation functions

Game playing demonstrates how AI systems can **analyze complex situations, predict outcomes, and make optimal strategic decisions**.

6.1 Introduction to Game Playing

Game Playing is an important application of Artificial Intelligence where an AI system learns to **make optimal decisions in competitive environments**. In these environments, two or more players compete, and each player tries to **maximize their chances of winning while minimizing the opponent's chances**.

AI game-playing programs analyze possible moves and predict future outcomes using **search algorithms and evaluation strategies**. According to the syllabus section for Unit-6, game playing focuses on **Minimax search, Alpha-Beta pruning, refinements, and iterative deepening** used in strategic games.

1. Concept of AI in Games

Meaning : In AI, a game is considered a **search problem where the system must explore different possible moves to find the best strategy**.

An AI game-playing system:

- analyzes possible moves
- predicts opponent responses
- chooses the optimal action

The AI agent uses **game trees and search algorithms** to determine the best move.

Example : Consider **Tic-Tac-Toe**.

Possible moves:

Current Board

X | O | X

O | X |

| | O

The AI system evaluates possible next moves and chooses the move that **leads to the highest chance of winning**.

Goal of AI in Games

Objective	Explanation
Win the game	choose best strategy
Predict opponent moves	anticipate opponent actions
Optimize decisions	select move with highest benefit

2. Types of Games in AI

Games in Artificial Intelligence are classified based on **environment characteristics and number of players**.

1. Deterministic vs Stochastic Games

Type	Description	Example
Deterministic	outcome determined by current state	chess
Stochastic	randomness involved	backgammon

2. Perfect Information vs Imperfect Information

Type	Description	Example
Perfect Information	all players know entire game state	chess
Imperfect Information	players have incomplete information	poker

3. Two-Player vs Multi-Player Games

Type	Description	Example
Two-player	two competing agents	chess
Multi-player	more than two players	multiplayer board games

4. Zero-Sum vs Non-Zero-Sum Games

Type	Description	Example
Zero-sum	one player's gain equals another's loss	chess
Non-zero-sum	multiple players may benefit	cooperative games

Examples of AI Game Applications

Game	AI Application
Chess	strategic decision making
Go	deep search algorithms
Tic-Tac-Toe	basic game tree search
Checkers	heuristic evaluation

3. Characteristics of Game Environments

Game environments in AI have specific characteristics that affect how AI agents make decisions.

1. Adversarial Environment

In games, players **compete against each other**.

Example:

- player tries to maximize score

- opponent tries to minimize it

2. Sequential Decisions

Game decisions are **sequential**, meaning each move affects future states.

Example: Move 1 → Move 2 → Move 3

Each step changes the game situation.

3. Strategic Planning

Players must plan **several moves ahead** to achieve victory.

AI uses search algorithms to evaluate **future possibilities**.

4. Finite State Space

Games have a **limited number of possible states**, although this number can be very large.

Example:

Game	Possible States
Tic-Tac-Toe	~255,000
Chess	~10 ¹²⁰ possible positions

4. How AI Agents Play Strategic Games

AI agents play strategic games using **search and evaluation techniques**.

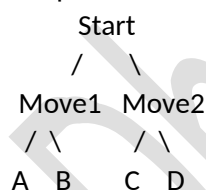
Step-by-Step Process

Step	Description
1	represent game states
2	generate possible moves
3	create game tree
4	evaluate possible outcomes
5	select best move

Game Tree Concept

A **game tree** represents possible moves in a game.

Example:



Each branch represents a possible move.

Strategy Used by AI

AI uses algorithms such as:

Algorithm	Purpose
Minimax	optimal decision making
Alpha-Beta pruning	reduce search complexity
Iterative deepening	improve search efficiency

These methods help AI evaluate **future game states efficiently**.

Example: Chess AI

Chess AI evaluates:

- piece positions
- possible moves
- opponent strategies

Then selects the move with the **highest winning probability**.

Real-World AI Game Systems

AI System	Game
Deep Blue	chess
AlphaGo	Go
OpenAI Five	Dota 2

These systems demonstrate how AI can perform **advanced strategic reasoning**.

Importance of Game Playing in AI

Game playing is important because it helps researchers study:

AI Concept	Application
Search algorithms	game trees
Decision making	optimal moves
Strategy planning	long-term planning

Game environments are useful for testing **intelligent decision-making algorithms**.

Summary

Concept	Meaning
Game Playing	AI decision making in competitive environments
Types of Games	deterministic, stochastic, zero-sum, imperfect information
Game Environment	adversarial, sequential, strategic
AI Strategy	search algorithms and evaluation functions

Game playing demonstrates how AI systems can **analyze complex situations, predict outcomes, and make optimal strategic decisions**.

6.3 Minimax Search Procedure

The **Minimax algorithm** is a decision-making algorithm used in Artificial Intelligence for **two-player adversarial games** such as chess, tic-tac-toe, and checkers.

The idea behind Minimax is that **one player tries to maximize the score while the opponent tries to minimize it**. The algorithm assumes that the opponent always plays optimally.

Using this assumption, the AI searches through the **game tree** and chooses the move that provides the **best guaranteed outcome**.

Overview

Concept	Meaning
Minimax	algorithm used to choose best move in competitive games
Maximizing Player	player trying to maximize score
Minimizing Player	opponent trying to minimize score
Game Tree Search	exploring possible moves
Optimal Strategy	selecting best guaranteed move

1. Minimax Concept

Meaning : The **Minimax algorithm** works by exploring possible future moves and calculating the **best possible outcome assuming the opponent also plays optimally**.

The algorithm works on the principle:

Maximize your minimum possible loss

This means the AI selects the move that **maximizes the minimum payoff**.

Basic Idea

In a two-player game:

- Player 1 tries to **maximize the score**
- Player 2 tries to **minimize the score**

The AI assumes the opponent will always choose the move that **reduces the AI's advantage**.

Example Terminal states with utility values:

Outcome	Utility Value
Win	+1
Draw	0
Loss	-1

The AI selects the move with the **highest possible minimum utility**.

2. Maximizing Player

The **maximizing player (MAX)** is the AI agent trying to **increase the utility value**.

Goal of MAX player: Choose move with maximum score

Example

Possible outcomes:

Move	Utility
Move A	+3
Move B	+5
Move C	+2

MAX player chooses: Move B

because it has the **highest utility value**.

3. Minimizing Player

The **minimizing player (MIN)** is the opponent trying to **reduce the AI's score**.

Goal of MIN player: Choose move with minimum score

Example

Possible outcomes:

Move	Utility
Move A	+4
Move B	+1
Move C	+3

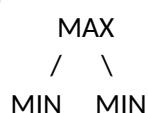
MIN player chooses:

Move B

because it gives the **lowest score for the AI**.

4. Minimax Game Tree Example

Example tree:



$$\begin{array}{cc}
 / \quad \backslash & / \quad \backslash \\
 3 \quad 5 & 2 \quad 9
 \end{array}$$

Step 1: MIN level chooses minimum values

Left MIN $\rightarrow \min(3,5) = 3$

Right MIN $\rightarrow \min(2,9) = 2$

Step 2: MAX level chooses maximum value

$\max(3,2) = 3$

Final decision:

Choose left branch

5. Minimax Algorithm Steps

The Minimax algorithm follows these steps.

Step	Description
1	construct game tree
2	assign utility values to terminal states
3	propagate values upward
4	MAX selects highest value
5	MIN selects lowest value

Algorithm Flow

Generate Game Tree

↓

Evaluate Terminal Nodes

↓

MIN chooses minimum values

↓

MAX chooses maximum value

↓

Best move selected

Simple Minimax Algorithm (Conceptual)

function MINIMAX(node, depth, maximizingPlayer):

if node is terminal or depth = 0

 return utility(node)

if maximizingPlayer

 best = $-\infty$

 for each child

 best = max(best, MINIMAX(child))

 return best

else

 best = $+\infty$

 for each child

 best = min(best, MINIMAX(child))

 return best

6. Optimal Decision-Making in Adversarial Games

Minimax helps AI make **optimal decisions in competitive environments**.
The algorithm ensures that the AI selects the move that **guarantees the best possible outcome even if the opponent plays perfectly**.

Example: Tic-Tac-Toe AI

Possible moves:

X | O | X

O | X |

| | O

The AI evaluates all possible moves and chooses the move that **leads to the best outcome**.

Applications of Minimax

Game	Application
Chess	move selection
Tic-Tac-Toe	optimal strategy
Checkers	board evaluation
Go	game tree search

Limitations of Minimax

The major limitation is **large search space**.

Example:

Game	Possible States
Tic-Tac-Toe	manageable
Chess	extremely large (~10 ¹²⁰)

Searching the entire tree becomes computationally expensive.

To solve this problem, AI uses:

- **Alpha-Beta pruning**
- heuristic evaluation functions

Summary

Concept	Meaning
Minimax	algorithm for adversarial game decision-making
MAX Player	tries to maximize score
MIN Player	tries to minimize score
Utility Value	numerical value of game outcome
Goal	choose best move assuming opponent plays optimally

The Minimax algorithm allows AI systems to **make optimal strategic decisions by analyzing future game states and anticipating opponent moves**.

6.4 Alpha-Beta Pruning

Alpha-Beta Pruning is an optimization technique used to improve the efficiency of the **Minimax algorithm**. It reduces the number of nodes that need to be evaluated in a game tree.
Instead of exploring every possible branch of the tree, Alpha-Beta pruning **eliminates branches that cannot affect the final decision**. This significantly reduces the computation time.
The final result of Minimax **does not change**, but the search becomes much faster.

Overview

Concept	Meaning
Alpha-Beta Pruning	optimization technique for Minimax
Pruning	removing unnecessary branches
Alpha Value	best value found for maximizing player
Beta Value	best value found for minimizing player
Goal	reduce number of nodes evaluated

1. Concept of Pruning

Meaning : Pruning means **cutting off branches of the game tree that do not influence the final decision.**

If a move is clearly worse than a previously examined move, the algorithm **does not evaluate that branch further.**

Why Pruning is Needed

Game trees can become extremely large.

Example:

Game	Approximate States
Tic-Tac-Toe	thousands
Chess	about 10^{120} states

Evaluating every node is computationally expensive.

Pruning helps by **ignoring useless branches.**

Example of Pruning



If MAX already has a better value from another branch, the algorithm may **skip evaluating some nodes.**

This process is called **pruning.**

2. Alpha Value

Meaning : Alpha (α) represents the **best value that the maximizing player can guarantee so far.** It acts as a **lower bound** for the maximizing player.

Representation

α = best value for MAX

Example

If MAX finds values: 3, 5

Then: $\alpha = 5$

This means the maximizing player **will not accept a value lower than 5.**

3. Beta Value

Meaning : Beta (β) represents the **best value that the minimizing player can guarantee so far.** It acts as an **upper bound** for the minimizing player.

Representation

β = best value for MIN

Example

If MIN finds values: 2, 4

Then: $\beta = 2$

This means the minimizing player **will not allow a value greater than 2**.

4. Pruning Condition

Pruning occurs when:

$$\alpha \geq \beta$$

This means:

- the maximizing player already has a better option
- the minimizing player will avoid this branch

So exploring the branch **cannot change the final result**.

Example

Game tree:



Step 1: Evaluate first MIN node

$$\min(3, 5) = 3$$

So: $\alpha = 3$

Step 2: Evaluate second MIN node

First value: 2

Now:

$$\beta = 2$$

Since:

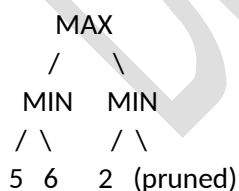
$$\alpha \geq \beta$$

$$3 \geq 2$$

Remaining nodes **do not need to be evaluated**.

Branch is **pruned**.

Example with Pruning



Explanation:

- MIN finds value 2
- MAX already has 5

Since **2 is worse**, remaining branch is ignored.

5. Alpha-Beta Algorithm Idea

The algorithm works similarly to Minimax but keeps track of **alpha and beta values during search**.

Algorithm Steps

Step	Description
1	start exploring game tree
2	update alpha value for MAX nodes
3	update beta value for MIN nodes
4	check pruning condition
5	skip branches when $\alpha \geq \beta$

Simplified Algorithm

function AlphaBeta(node, depth, α , β , maximizingPlayer):

if node is terminal
 return utility(node)

if maximizingPlayer
 for each child
 $\alpha = \max(\alpha, \text{AlphaBeta}(\text{child}))$
 if $\alpha \geq \beta$
 break (prune)
 return α

else
 for each child
 $\beta = \min(\beta, \text{AlphaBeta}(\text{child}))$
 if $\beta \leq \alpha$
 break (prune)
 return β

Effect of Alpha-Beta Pruning

Without pruning: AI evaluates **all nodes** in the game tree.

With pruning: Many nodes are skipped.

Example

Method	Nodes Evaluated
Minimax	100% nodes
Alpha-Beta	significantly fewer nodes

In many cases, Alpha-Beta pruning can **reduce search complexity dramatically**.

Advantages of Alpha-Beta Pruning

Advantage	Explanation
Faster search	fewer nodes evaluated
Same result as Minimax	optimal decision maintained
Efficient for large games	useful for chess and similar games

Limitations

Limitation	Explanation
depends on move ordering	better ordering improves pruning

still expensive for large games	deep trees require more computation
---------------------------------	-------------------------------------

Applications

Game	Usage
Chess AI	reduce search space
Checkers	improve decision speed
Game engines	strategic move selection

Many modern game AI systems combine **Minimax with Alpha-Beta pruning**.

Summary

Concept	Meaning
Alpha-Beta Pruning	optimization technique for Minimax
Alpha	best value for maximizing player
Beta	best value for minimizing player
Pruning Condition	$\alpha \geq \beta$
Goal	reduce number of evaluated nodes

Alpha-Beta pruning improves the efficiency of game-playing AI by **eliminating unnecessary branches while preserving optimal decisions**.

6.5 Refinements of Game Tree Search

In complex games like chess or Go, the **game tree becomes extremely large**. Searching every possible move using Minimax is often impractical because it requires too much computation. To solve this problem, several **refinement techniques** are used to make game tree search more efficient. These techniques improve the performance of search algorithms by **reducing the number of nodes explored and focusing on promising moves**.

Two important refinement methods are:

- **Move Ordering**
- **Heuristic Evaluation Functions**

Overview

Concept	Meaning
Refinement	improvement of search algorithms
Move Ordering	evaluating promising moves first
Heuristic Evaluation	estimating value of non-terminal states
Goal	improve search efficiency

1. Move Ordering

Meaning: **Move ordering** is a technique where the AI system evaluates **more promising moves first** in the search process.

If good moves are explored earlier, **Alpha-Beta pruning becomes more effective**, and many unnecessary branches can be pruned quickly.

Why Move Ordering is Important

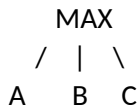
In Alpha-Beta pruning, the earlier a strong move is discovered, the sooner other branches can be eliminated.

Better move ordering leads to:

- fewer nodes explored
- faster decision-making

Example

Game tree:



If move **B** is the best move but explored last:

- many unnecessary nodes will be evaluated.

If move **B** is evaluated first:

- pruning occurs earlier
- fewer nodes are explored.

Example in Chess

AI prioritizes moves such as:

Move Type	Importance
Capturing opponent piece	high priority
Check move	high priority
Normal move	lower priority

Exploring these moves first improves search efficiency.

Techniques for Move Ordering

Technique	Description
Capture moves first	prioritize attacks
Check moves first	evaluate threats
History heuristic	prioritize previously successful moves

These methods help the AI focus on **important moves earlier in the search**.

2. Heuristic Evaluation Functions

Meaning : A **heuristic evaluation function** estimates the **value of a game state when the search cannot reach a terminal state**.

In large games, exploring the entire tree is impossible. Instead, the AI stops at a certain depth and uses heuristics to estimate how good the position is.

Why Heuristics are Needed

Many games have extremely large search spaces.

Example:

Game	Approximate Positions
Tic-Tac-Toe	manageable
Chess	about 10^{120} states

It is impossible to evaluate every possible state, so AI uses **heuristic evaluation**.

Example

Chess evaluation function:

Evaluation Score =

Material Advantage

+ Position Advantage

+ King Safety

Each factor contributes to the final score.

Example Evaluation Table

Factor	Score
Queen	+9
Rook	+5
Bishop	+3
Knight	+3
Pawn	+1

If a player has more valuable pieces, the evaluation score increases.

Example Heuristic Score

Position evaluation:

Player	Score
AI	+5
Opponent	+3

Evaluation:

Score = +2

This means the AI has an advantage.

Depth-Limited Search

To apply heuristics, the search stops at a certain **depth limit**.

Example:

Depth 0 → current state

Depth 1 → next move

Depth 2 → opponent move

Depth 3 → heuristic evaluation

The heuristic function estimates the utility value.

Example Game Tree with Heuristic Evaluation



These values come from the **heuristic evaluation function**.

The AI uses them to determine the best move.

Advantages of Heuristic Evaluation

Advantage	Explanation
Reduces search depth	avoids exploring entire tree
Faster decisions	evaluates positions quickly
Handles complex games	useful for large search spaces

Limitations

Limitation	Explanation
approximation	evaluation may not be perfect
domain knowledge required	heuristics depend on game expertise

Combining Refinements

Modern AI game systems combine multiple techniques:

Technique	Purpose
Minimax	optimal decision making
Alpha-Beta pruning	reduce nodes
Move ordering	improve pruning
Heuristic evaluation	estimate non-terminal states

Together they make game-playing AI **much more efficient**.

Example: Chess AI Strategy

Process used by many chess engines:

Generate Moves



Order Moves



Apply Minimax



Use Alpha-Beta Pruning



Evaluate Positions using Heuristics

This approach allows AI to search **millions of positions efficiently**.

Summary

Concept	Meaning
Refinements	techniques to improve search efficiency
Move Ordering	evaluate promising moves first
Heuristic Evaluation	estimate value of game states
Goal	reduce search complexity

Refinements of game tree search help AI systems **handle large game trees efficiently while maintaining strong strategic decision-making**.

6.6 Iterative Deepening

This technique combines the advantages of **depth-first search** and **breadth-first search** to improve game tree exploration.

6.6 Iterative Deepening: Concept of iterative, deepening search, depth limits,Combining depth-first search with breadth exploration

6.6 Iterative Deepening : Iterative Deepening is a search technique used in Artificial Intelligence to explore game trees efficiently. It combines the advantages of **Depth-First Search (DFS)** and **Breadth-First Search (BFS)**.

Instead of searching the entire game tree at once, the algorithm **repeatedly performs depth-limited searches with increasing depth limits**. This allows the AI to gradually explore deeper levels of the game tree.

Iterative deepening is widely used in **game-playing algorithms** such as **Minimax** and **Alpha-Beta pruning**.

Overview

Concept	Meaning
Iterative Deepening	repeated search with increasing depth
Depth Limit	maximum depth explored in search

Depth-Limited Search	DFS restricted to a fixed depth
Goal	explore game tree efficiently

1. Concept of Iterative Deepening

Meaning : Iterative deepening performs **multiple depth-limited searches**, increasing the depth limit in each iteration.

The algorithm starts with shallow searches and gradually increases the depth until the goal is found.

Example

Search depths:

Depth = 1

Depth = 2

Depth = 3

Depth = 4

Each iteration explores deeper levels of the game tree.

Example Search Process

Iteration 1 → search depth 1

Iteration 2 → search depth 2

Iteration 3 → search depth 3

Iteration 4 → search depth 4

The search continues until the desired depth or solution is reached.

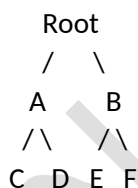
2. Depth-Limited Search

Meaning : A **depth-limited search** is a form of depth-first search where the exploration stops at a predetermined depth.

This prevents the algorithm from exploring extremely deep branches of the tree.

Example

Game tree:



If depth limit = 2, the search stops at nodes: C, D, E, F

Nodes beyond that depth are not explored.

3. Iterative Deepening Algorithm

The algorithm repeatedly increases the depth limit.

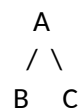
Steps

Step	Description
1	set depth limit = 1
2	perform depth-limited search
3	if goal not found increase depth
4	repeat search with larger depth
5	continue until solution found

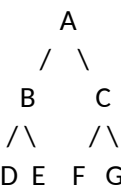
Example Tree Exploration

Depth 1: A

Depth 2



Depth 3



Each iteration expands the tree further.

Combining Depth-First and Breadth-First Search

Iterative deepening combines the strengths of DFS and BFS.

Breadth-First Search Characteristics

Feature	Description
Search order	level by level
Memory usage	high
Solution guarantee	optimal solution

Depth-First Search Characteristics

Feature	Description
Search order	deep into tree
Memory usage	low
Solution guarantee	not always optimal

Iterative Deepening Advantages

Feature	Explanation
Memory efficient	similar to DFS
Optimal solution	similar to BFS
Gradual exploration	explores tree level by level

Example of Iterative Deepening Search

Iteration 1 : A

Iteration 2

A → B, C

Iteration 3

A → B → D, E

A → C → F, G

The algorithm gradually explores deeper nodes.

Use in Game Playing

In game-playing AI, iterative deepening is used with **Minimax and Alpha-Beta pruning**.

This allows the AI to:

- search deeper when more time is available
- return best move found so far if time runs out

Example: Chess AI

Search process:

Depth 1 → evaluate immediate moves

Depth 2 → evaluate opponent responses

Depth 3 → evaluate future positions

The algorithm gradually improves decision quality.

Advantages of Iterative Deepening

Advantage	Explanation
efficient memory usage	similar to DFS
guarantees optimal solution	like BFS
flexible depth control	good for time-limited search

Limitations

Limitation	Explanation
repeated exploration	nodes may be visited multiple times
extra computation	repeated depth searches

However, the benefits often outweigh these costs.

Applications

Application	Example
Game AI	chess engines
Robotics	path planning
Search problems	puzzle solving

Summary

Concept	Meaning
Iterative Deepening	repeated depth-limited search
Depth Limit	maximum search depth
Depth-Limited Search	DFS restricted by depth
Goal	efficient exploration of search tree

Iterative deepening helps AI systems **search deeper levels of the game tree efficiently while maintaining manageable memory usage**.